CS 101 - Algorithms & Programming I

Spring 2026 - Lab 7

Due: Week of April 14, 2026

*Remember the **honor code** for your programming assignments.*
*For all labs, your solutions must conform to the CS101 style **guidelines**!*
All data and results should be stored in variables (or constants where appropriate) with meaningful names.

The objective of this lab is to learn how to define and use custom classes. Remember that analyzing your problems and designing them on a piece of paper *before* starting implementation/coding is always a best practice. Specifically for this lab, you are to organize your data and methods, working on them as classes.

0. Setup Workspace

Start VSC and open the previously created workspace named `labs_ws`. Now, under the `labs` folder, create a new folder named `lab7`.

In this lab, you are to create Java classes/files (under `labs/lab7` folder) as described below. You are expected to submit 4 files for your initial solution (`MinesweeperGame, GameController, Board, and Cell`). **Please submit the files without compressing them, and ensure that no other or previous lab solutions are included.**

1. Minesweeper Game

Hidden beneath a quiet grid lies danger. The board looks innocent — a field of unrevealed cells — but concealed within are **mines** waiting to be triggered. The player's goal is simple: reveal every safe cell without detonating a single mine. Each revealed cell shows a **hint number** indicating how many mines are hidden in its 8 neighboring cells. Use logic, patience, and the hints to navigate the minefield. One wrong move and it's over.

image credit: https://minesweeper.online/

In this exercise, you will use object-oriented programming concepts to implement the **Minesweeper Game**. All class structures are provided, along with some partial implementations. You are expected to complete the remaining parts of the code, and you may add helper methods if needed.

In Minesweeper game, there is a grid of hidden cells, come containing mines. Each cell is either containing mine or safe. If there is a bomb on a neighboring cells of a safe cell, the number of the neighboring mines is written in the cell. If there is no neighboring mine, the cell is revealed as empty. On each turn, the player either reveals a cell or places a flag on a suspected mine. Revealing a safe cell uncovers a hint number showing how many of its up to 8 neighboring cells contain mines; if that number is zero, all neighboring cells are automatically revealed in a chain. Revealing a mine finishes the game failing the player. Putting a flag only marks that cell suspecting there is a mine in that cell. If a cell is flagged, player cannot reveal that cell until player removing the flag. The player wins by successfully revealing every safe cell without revealing a mine.

In the game, firstly, 8X10 grid of hidden cells is shown. In each turn, the player is asked if they put a flag or reveal a cell, and the indexes of the cell. If the player gives an invalid index, the program asks until a valid index. If the player selects flag, the cell is marked with `">"` and in the grid the cell is shown as `">"`. If the player to put flag on already flagged cell, flag is removed. `Scores` show the number of revealed cells. `Flags` shows the number of flags placed. `Mines remaining` shows the (number of mines - placed flags).

`MinesweeperGame` Class:

This class handles the overall game flow and user interactions. It runs the main game loop, renders the board, handles player input for revealing and flagging cells, and displays end-of-game messages. It also stores static variables shared across the project. The class focuses on controlling the *sequence* of the game, not on managing internal state and logic — that is handled by `GameController`.

Static Variables:

- `ROWS`: Number of rows in the minesweeper grid.
- `COLS`: Number of columns in the minesweeper grid.
- `TOTAL_MINES`: Total number of mines hidden on the board.
- `HIDDEN_SYMBOL`: Symbol for an unrevealed cell. ("#")
- `MINE_SYMBOL`: Symbol for a mine, shown on game over. ("O")
- `FLAG_SYMBOL`: Symbol for a player-placed flag. (">")
- `EMPTY_SYMBOL`: Symbol for a revealed cell with no neighboring mines. (" ")
- `VICTORY_MESSAGE`: Message displayed when the player wins.
- `DEFEAT_MESSAGE`: Message displayed when the player hits a mine.
- `gameController`: Instance of GameController managing game logic.
- `scanner`: Instance of Scanner for reading player input.

Methods:

- `main(String[] args)`: Entry point. Calls `initializeVariables()`, then `playGame()`, then `handleGameEnding()`.
- `initializeVariables()`: Creates instances of Scanner and GameController.
- `playGame()`: Main game loop. Continues until the game is over. Each iteration renders the board, then asks the player to choose an action (reveal or flag) and a cell coordinate.
- `handleGameEnding()`: Renders the final board (mines revealed), prints the victory or defeat message and the player's final score, then closes the scanner.
- `renderBoard()`: Prints the current board state to the console with row and column indices for reference.
- `renderGameInformation()`: Prints the player's current score, number of flags placed, and mines remaining.
- `handlePlayerAction()`: Prompts the player to choose an action: (R)eveal or (F)lag. Reads a valid row and column, then calls the appropriate method on GameController.
- `getValidInput(String prompt, int min, int max)`: Reads and validates an integer from the player, repeating until a value within [min, max] is entered.

`GameController` Class:

This class manages the internal game state and core logic. It holds the Board, tracks the player's score and flag count, processes player actions, and determines whether the game has ended. It focuses on game rules and state management, independent of rendering or player input.

Instance Variables:

- `board`: The Board object representing the current game grid.
- `score`: The player's current score. Increases by 1 for every safe cell successfully revealed.
- `flagsPlaced`: Number of flags the player has placed on the board.
- `gameOver`: True if the player has hit a mine.

Constructor & Methods:

- `GameController()`: Initializes a new Board, sets score and flagsPlaced to 0, and sets gameOver to false.
- `handleReveal(int row, int col)`: Attempts to reveal the cell at (row, col). If the cell is already revealed or flagged, does nothing. If a mine is hit, sets gameOver to true and calls `board.revealAllMines()`. Otherwise increments score by the number of newly revealed cells.
- `handleFlag(int row, int col)`: Toggles the flag at (row, col). If a flag is added, increments flagsPlaced; if removed, decrements it. Clamps `flagsPlaced` to 0.
- `isGameOver()`: Returns true if the player hit a mine (`gameOver` is true) or all safe cells are revealed.
- `isVictory()`: Returns true if all safe cells are revealed and `gameOver` is false.
- `getScore()`: Returns the current score.
- `getFlagsPlaced()`: Returns the number of flags placed.
- `getMinesRemaining()`: Returns `TOTAL_MINES` - flagsPlaced (can be negative if over-flagged).
- `getBoard()`: Returns the Board object.

`Board` Class:

This class represents the full minesweeper grid. It manages a 2-D array of `Cell` objects, handles mine placement, computes neighbor mine counts, and provides methods to reveal cells — including a **flood-fill** that automatically reveals all connected empty cells when a zero-hint cell is uncovered

Instance Variables:

- `grid`: A 2-D array of `Cell` objects with dimensions `ROWS × COLS`.

Constructor & Methods:

- `Board()`: Initializes the grid by creating a Cell for every position, placing mines
- `initializeGrid()`: Creates a new Cell at every (row, col) position in the grid.
- `placeMines()`: Randomly selects TOTAL_MINES distinct cells and marks them as mines using `setMine(true)`. Uses a loop that keeps picking random positions until enough unique mines have been placed.
- `computeNeighborCounts()`: Iterates over every cell and sets its neighborMineCount by counting how many of its up-to-8 neighbors are mines.
- `getNeighbors(int row, int col)`: Returns an ArrayList of all valid neighboring Cell objects for the cell at (row, col). Checks bounds carefully to avoid IndexOutOfBoundsException.
- `revealCell(int row, int col)`: Reveals the cell at (row, col). If it is a mine, returns false (game over). If its neighbor count is 0, triggers `floodReveal()` to auto-reveal connected empty cells. Returns true if no mine was hit.
- `floodReveal(int row, int col)`: Reveals all connected unrevealed, non-mine cells that have a neighbor count of 0, and also reveals the border cells (neighbor count > 0) that surround the empty region.
- `flagCell(int row, int col)`: Toggles the flag on the cell at (row, col) if it is within bounds.
- `revealAllMines()`: Reveals every mine on the board. Called when the game ends in defeat so the player can see where all mines were.
- `allSafeCellsRevealed()`: Returns true if every non-mine cell has been revealed — the victory condition.
- `getCell(int row, int col)`: Returns the Cell at (row, col), or null if out of bounds.
- `getGrid()`: Returns the full 2-D Cell array.

`Cell` Class:

This class represents a single cell on the Minesweeper board. Each cell knows whether it contains a mine, whether it has been revealed, whether it has been flagged by the player, and how many mines are among its neighbors.

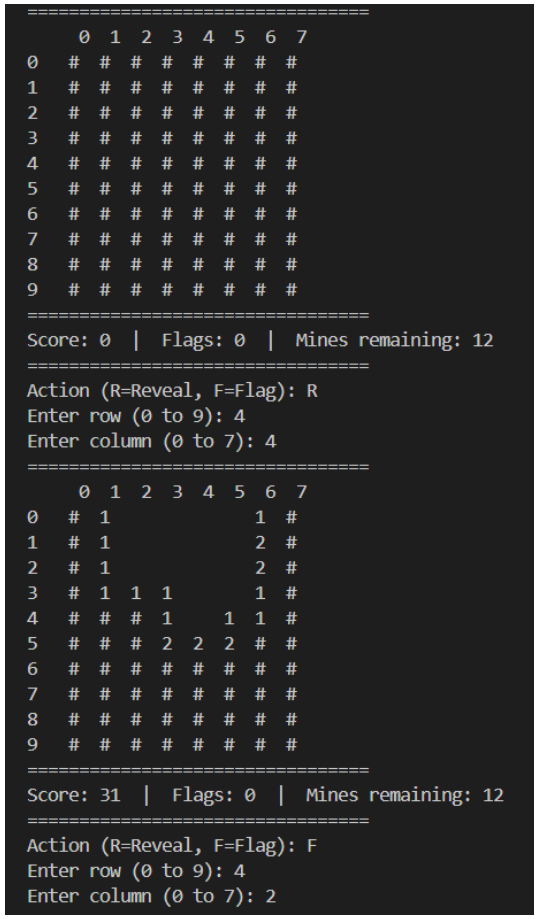
Instance Variables:

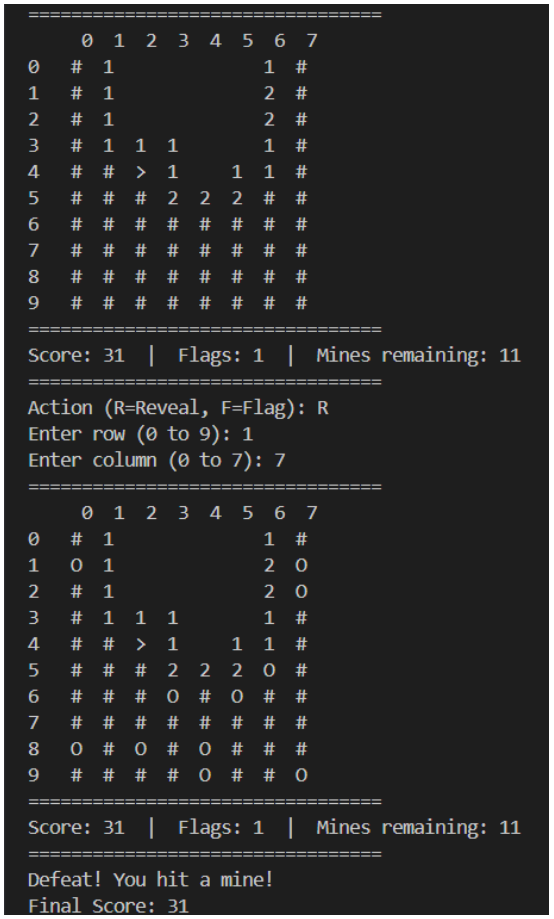
- `isMine`: True if this cell contains a mine.
- `isRevealed`: True if this cell has been revealed by the player.
- `isFlagged`: True if the player has placed a flag on this cell.
- `neighborMineCount`: Number of mines in the 8 surrounding cells (0–8). Set after the board is fully initialized.

Constructor & Methods:

- `Cell()`: Initializes a non-mine, unrevealed, unflagged cell with a neighbor count of 0.
- `reveal()`: Marks the cell as revealed. Has no effect if the cell is flagged.
- `toggleFlag()`: Places or removes a flag on an unrevealed cell. Has no effect if the cell is already revealed.
- `display()`: Returns the string symbol that should be printed for this cell:
  - If flagged → FLAG_SYMBOL
  - If not revealed → HIDDEN_SYMBOL
  - If revealed and mine → MINE_SYMBOL
  - If revealed and neighborMineCount > 0 → the digit as a string
  - If revealed and neighborMineCount == 0 → EMPTY_SYMBOL
- `isMine()`: Returns whether this cell is a mine.
- `isRevealed()`: Returns whether this cell has been revealed.
- `isFlagged()`: Returns whether this cell is flagged.
- `getNeighborMineCount()`: Returns the neighbor mine count.
- `setMine(boolean mine)`: Sets whether this cell is a mine.
- `setNeighborMineCount(int count)`: Sets the neighbor mine count.

Gameplay:


0 1 2 3 4 5 6 7
0 # # # # # # # #
1 # # # # # # # #
2 # # # # # # # #
3 # # # # # # # #
4 # # # # # # # #
5 # # # # # # # #
6 # # # # # # # #
7 # # # # # # # #
8 # # # # # # # #
9 # # # # # # # #
Score: 0 | Flags: 0 | Mines remaining: 12
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 4
Enter column (0 to 7): 4
0 1 2 3 4 5 6 7
0 # 1 1 #
1 # 1 2 #
2 # 1 2 #
3 # 1 1 1 1 #
4 # # # 1 1 1 #
5 # # # 2 2 2 # #
6 # # # # # # # #
7 # # # # # # # #
8 # # # # # # # #
9 # # # # # # # #
Score: 31 | Flags: 0 | Mines remaining: 12
Action (R=Reveal, F=Flag): F
Enter row (0 to 9): 4
Enter column (0 to 7): 2


0 1 2 3 4 5 6 7
0 # 1 1 #
1 # 1 2 #
2 # 1 2 #
3 # 1 1 1 1 #
4 # # > 1 1 1 #
5 # # # 2 2 2 # #
6 # # # # # # # #
7 # # # # # # # #
8 # # # # # # # #
9 # # # # # # # #
Score: 31 | Flags: 1 | Mines remaining: 11
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 1
Enter column (0 to 7): 7
0 1 2 3 4 5 6 7
0 # 1 1 #
1 O 1 2 O
2 # 1 2 O
3 # 1 1 1 1 #
4 # # > 1 1 1 #
5 # # # 2 2 2 O #
6 # # # O # O # #
7 # # # # # # # #
8 O # O # O # # #
9 # # # # O # # O
Score: 31 | Flags: 1 | Mines remaining: 11
Defeat! You hit a mine!
Final Score: 31

```
==============================
    0 1 2 3 4 5 6 7
0   # # # # # # # #
1   # # # # # # # #
2   # # # # # # # #
3   # # # # # # # #
4   # # # # # # # #
5   # # # # # # # #
6   # # # # # # # #
7   # # # # # # # #
8   # # # # # # # #
9   # # # # # # # #
==============================
Score: 0  |  Flags: 0  |  Mines remaining: 12
==============================
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 4
Enter column (0 to 7): 4
==============================
    0 1 2 3 4 5 6 7
0   # # # # # # 1
1   1 1 1 # # # 1
2       1 3 # 3 1
3         1 1 1
4
5   1 2 2 2 2 2 1
6   # # # # # # 1
7   # # # # # # 1
8   # # # # # # 1 1
9   # # # # # # # #
==============================
Score: 44  |  Flags: 0  |  Mines remaining: 12
==============================
Action (R=Reveal, F=Flag): F
Enter row (0 to 9): 2
Enter column (0 to 7): 4
```

```
==============================
    0 1 2 3 4 5 6 7
0   # # # # # # 1
1   1 1 1 # # # 1
2       1 3 > 3 1
3         1 1 1
4
5   1 2 2 2 2 2 1
6   # # # # # # 1
7   # # # # # # 1
8   # # # # # # 1 1
9   # # # # # # # #
==============================
Score: 44  |  Flags: 1  |  Mines remaining: 11
==============================
Action (R=Reveal, F=Flag): F
Enter row (0 to 9): 2
Enter column (0 to 7): 4
==============================
    0 1 2 3 4 5 6 7
0   # # # # # # 1
1   1 1 1 # # # 1
2       1 3 # 3 1
3         1 1 1
4
5   1 2 2 2 2 2 1
6   # # # # # # 1
7   # # # # # # 1
8   # # # # # # 1 1
9   # # # # # # # #
==============================
Score: 44  |  Flags: 0  |  Mines remaining: 12
==============================
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 0
Enter column (0 to 7): 4
```

```
==============================
    0 1 2 3 4 5 6 7
0   # # # # 3 # 1
1   1 1 1 # # # 1
2       1 3 # 3 1
3         1 1 1
4
5   1 2 2 2 2 2 1
6   # # # # # # 1
7   # # # # # # 1
8   # # # # # # 1 1
9   # # # # # # # #
==============================
Score: 45  |  Flags: 0  |  Mines remaining: 12
==============================
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 0
Enter column (0 to 7): 3
==============================
    0 1 2 3 4 5 6 7
0   # # # 2 3 # 1
1   1 1 1 # # # 1
2       1 3 # 3 1
3         1 1 1
4
5   1 2 2 2 2 2 1
6   # # # # # # 1
7   # # # # # # 1
8   # # # # # # 1 1
9   # # # # # # # #
==============================
Score: 46  |  Flags: 0  |  Mines remaining: 12
==============================
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 0
Enter column (0 to 7): 5
```

```
==============================
    0 1 2 3 4 5 6 7
0   # # # 2 3 2 1
1   1 1 1 # # # 1
2       1 3 # 3 1
3         1 1 1
4
5   1 2 2 2 2 2 1
6   # # # # # # 1
7   # # # # # # 1
8   # # # # # # 1 1
9   # # # # # # # #
==============================
Score: 47  |  Flags: 0  |  Mines remaining: 12
==============================
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 0
Enter column (0 to 7): 1
==============================
    0 1 2 3 4 5 6 7
0   # 1 # 2 3 2 1
1   1 1 1 # # # 1
2       1 3 # 3 1
3         1 1 1
4
5   1 2 2 2 2 2 1
6   # # # # # # 1
7   # # # # # # 1
8   # # # # # # 1 1
9   # # # # # # # #
==============================
Score: 48  |  Flags: 0  |  Mines remaining: 12
==============================
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 0
Enter column (0 to 7): 2
```

```
================================
    0  1  2  3  4  5  6  7
0   #  1  1  2  3  2  1
1   1  1  1  #  #  #  1
2         1  3  #  3  1
3            1  1  1
4
5   1  2  2  2  2  2  1
6   #  #  #  #  #  #  1
7   #  #  #  #  #  #  1
8   #  #  #  #  #  #  1  1
9   #  #  #  #  #  #  #  #
================================
Score: 49  |  Flags: 0  |  Mines remaining: 12
================================
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 8
Enter column (0 to 7): 2
================================
    0  1  2  3  4  5  6  7
0   #  1  1  2  3  2  1
1   1  1  1  #  #  #  1
2         1  3  #  3  1
3            1  1  1
4
5   1  2  2  2  2  2  1
6   #  #  #  #  #  #  1
7   #  3  2  3  #  #  1
8   1  1     1  1  2  1  1
9                  1  #  #
================================
Score: 64  |  Flags: 0  |  Mines remaining: 12
================================
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 7
Enter column (0 to 7): 5
```

```
================================
    0  1  2  3  4  5  6  7
0   #  1  1  2  3  2  1
1   1  1  1  #  #  #  1
2         1  3  #  3  1
3            1  1  1
4
5   1  2  2  2  2  2  1
6   #  #  #  #  #  #  1
7   #  3  2  3  #  3  1
8   1  1     1  1  2  1  1
9                  1  #  #
================================
Score: 65  |  Flags: 0  |  Mines remaining: 12
================================
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 9
Enter column (0 to 7): 7
================================
    0  1  2  3  4  5  6  7
0   #  1  1  2  3  2  1
1   1  1  1  #  #  #  1
2         1  3  #  3  1
3            1  1  1
4
5   1  2  2  2  2  2  1
6   #  #  #  #  #  #  1
7   #  3  2  3  #  3  1
8   1  1     1  1  2  1  1
9                  1  #  #
================================
Score: 66  |  Flags: 0  |  Mines remaining: 12
================================
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 6
Enter column (0 to 7): 3
```

```
================================
    0  1  2  3  4  5  6  7
0   #  1  1  2  3  2  1
1   1  1  1  #  #  #  1
2         1  3  #  3  1
3            1  1  1
4
5   1  2  2  2  2  2  1
6   #  #  #  3  #  #  1
7   #  3  2  3  #  3  1
8   1  1     1  1  2  1  1
9                  1  #  1
================================
Score: 67  |  Flags: 0  |  Mines remaining: 12
================================
Action (R=Reveal, F=Flag): R
Enter row (0 to 9): 6
Enter column (0 to 7): 0
================================
    0  1  2  3  4  5  6  7
0   #  1  1  2  3  2  1
1   1  1  1  #  #  #  1
2         1  3  #  3  1
3            1  1  1
4
5   1  2  2  2  2  2  1
6   2  #  #  3  #  #  1
7   #  3  2  3  #  3  1
8   1  1     1  1  2  1  1
9                  1  #  1
================================
Score: 68  |  Flags: 0  |  Mines remaining: 12
================================
Victory! You cleared the minefield!
Final Score: 68
```